Lab 1 - Osnove C# / LINQ

Predaja: Petak 3.4.2026.

Zadaci i bodovanje

Kriterij	Bodovi
Granularno izvođenje agenta po taskovima	1
Ispitivanje agenta za pojašnjenja	1
Usmeno ispitivanje LINQ/await ili C# koncepata	2
Objektni model projektnog zadatka	2

☐ Dodati javni GitHub repozitorij koji će biti za rad na projektu (sve slati na 'main' branch)

- ☐ Javiti na mail GitHub repo link **najkasnije do 20.3.**
- ☐ Sav kod koji se ocjenjuje mora biti na GH repozitoriju
- ☐ U root folder repozitorija dodati folder lab-1 gdje je potrebno uploadati "log" koristenja AI agenta

☐ Potrebno je kreirati objektni model prema vlastitom projektnom zadatku, koji se sastoji od barem 7 klasa, gdje 4 klase moraju biti kompleksne (preko 5 svojstava). Svojstva trebaju biti smislena i dobrog tipa:

- barem jedan vlastiti enum
- barem jedno DateTime svojstvo
- ispravne konekcije 1-N, N-N

☐ Potrebno je u main programu "napuniti" objekte podacima - kreirati razgranato barem 3 glavna objekta.

- primjeice, ako je tema aplikacije "Nogometni turnir", potrebno je imati barem 3 turnira s po barem 3 ekipe.

☐ U main programu je potrebno definirati smislene LINQ upite nad objektnim modelom koji će se kasnije (možda) koristiti u samoj aplikaciji

- ☐ Potrebno je razumjeti LINQ naredbe koje su dodane i moći ih modificirati uz pomoć AI agenta

- [ ] Proučiti i razumjeti koncept async-await
- [ ] Sav kod treba biti na GH **najkasnije do 2.4.2026. u 18:00**

Osnove C# jezika

U ovom će se poglavlju prezentirati osnovni pojmovi i koncepti koji će biti korišteni na kolegiju.

Solution i projekti

U pravilu se svaka ozbiljnija aplikacija dijeli na nekoliko slojeva. Ti slojevi su najčešće prezentacijski sloj, model/business/services sloj i sloj za pristup bazi podataka (Data Access Layer ili kraće DAL sloj). Kod razvoja višeslojnih aplikacija, potrebno je uvesti i nezavisnosti između slojeva u „smjeru prema gore", što znači da prezentacijski sloj može biti ovisan o DAL ili model sloju, ali DAL ili model sloj ne smiju biti ovisni o prezentacijskom sloju. U .NET okruženju se takva raspodjela najelegantnije postiže raspodjelom slojeva na projekte unutar rješenja (solutiona). Po uzoru na tu ideju, napravit ćemo novi projekt i nazvati ga Vjezba.Model. Novi projekt se dodaje odabirom opcije **File** → **Add** → **New Project** u izborniku.

Ako pogledamo solution explorer, novi projekt je kreiran i automatski mu je dodana nova klasa Class1, koju možemo slobodno obrisati.

Klase i objekti, svojstva u klasi

Nova klasa u projekt se dodaje tako da u solution explorer prozoru, na željenom projektu, odaberemo desni klik i add new item, te pronađemo odgovarajuću stavku iz izbornika.

> ⚠️ **Važna napomena**: pri dodavanju nove klase njena vidljivost je automatski postavljena na „internal", te da bi klasa bila vidljiva u drugim projektima potrebno ju je definirati kao „public".

Primjerice, ukoliko dodamo klasu **Fakultet** u projekt Vjezba.Model, moramo dodati ključnu riječ „public":

```
namespace Vjezba.Model
{
    public class Fakultet
    {
    }
}
```

Odnos klasa

Klase mogu implementirati nasljeđivanje, što znači da izvedena klasa nasljeđuje svojstva i funkcije iz bazne klase. Klase također mogu biti u odnosu u kojem neka klasa sadrži niz referenci na objekte druge klase, najčešće u obliku liste. U nastavku je dan primjer, gdje klasa A nasljeđuje klasu B (B je bazna klasa), dok klasa A također ima u sebi listu referenci na objekte klase C (tzv. odnos 1-N).

```
public class B
{
}

public class A : B
{
    private List<C> _objektiKlaseC;
}

public class C
{
}
```

Svojstva i polja, konstruktori i metode

Kada govorimo o informacijama koje su sadržane u klasama, najčešće koristimo termine kao što su svojstva (**properties**) i rijeđe polja (**fields**). Polje (član) je doslovno varijabla koja je sadržana unutar neke klase, i kad se kreira instanca neke klase, moguće je koristiti to polje kao i svaku drugu varijablu. Iz gornjeg isječka koda, član `_objektiKlaseC` je privatna varijabla, koja je tipa `List<C>`. U pravilu, članovi bi uvijek trebali biti privatni, što znači da se tim članovima ne može pristupiti nigdje izvan klase u kojoj se nalaze. Svojstvo, s druge strane,

možemo promatrati kao getter i setter funkciju, samo napisanu malo drukčijom sintaksom nego što je to u C++ ili Java jeziku:

```
public class Osoba
{
    private string _adresa;
    public string Adresa
    {
        get
        {
            return this._adresa;
        }
        set
        {
            this._adresa = value;
        }
    }
}
```

Ekvivalentno u Java/C++ stilu:

```
public class Osoba
{
    private string _adresa;

    public string GetAdresa()
    {
        return this._adresa;
    }

    public void SetAdresa(string value)
    {
        this._adresa = value;
    }
}
```

U gornjoj klasi, svojstvo je **Adresa**, a funkcije koje dohvaćaju ili postavljaju vrijednost tog svojstva su definirane ključnim riječima **get** i **set**. Vrijednost

svojstva se u pravilu čuva u privatnom članu. Što se tiče prava pristupa, vrijede ista pravila kao i za polja – međutim kod svojstava se najčešće koristi pravo pristupa **public** ili **protected**. Postoje i automatska svojstva:

```
public int SvojstvoX { get; set; }
```

Zašto koristiti automatska svojstva a ne polja? Postoje razlike kako C# doživljava varijable i svojstva. Držat ćemo se sljedećih pravila:

- Klasa ne smije sadržavati javna polja
- Klasa bi trebala sadržavati svojstva (javna ili protected, u rijetkim slučajevima private)
- Koristiti automatsko svojstvo gdje je moguće

Pri kreiranju objekta, prva funkcija koja se poziva unutar klase je **konstruktor**. Da bi klasa funkcionirala, nije potrebno napraviti konstruktor – osnovni (default) konstruktor će biti dodan automatski. Međutim, u nekim slučajevima je dobro dodati konstruktor za osiguravanje inicijalizacije:

```
public class A : B
{
    public List<C> ObjektiKlaseC { get; set; }

    public A()
    {
        ObjektiKlaseC = new List<C>();
    }
}
```

U gornjem slučaju je dobro koristiti konstruktor jer tada možemo inicijalizirati na ispravan način kompleksni objekt – kreirati mu novu instancu. Kada konstruktora ne bi bilo, onaj tko koristi tu klasu mora samostalno inicijalizirati listu, te ukoliko ju pokuša koristiti bez inicijalizacije, može dobiti **null reference** iznimku.

Uz konstruktore i svojstva, gotovo svaka klasa sadrži niz **metoda** – funkcija koje predstavljaju implementaciju ponašanja neke klase. Funkcija može imati razine pristupa **private**, **protected** i **public**, a može biti zavisna od postojanja objekta ili nezavisna (**static**).

```
public int PrebrojiKolikoJeUListi()
{
    int rezultat = 0;
    foreach (var x in ObjektiKlaseC)
        rezultat++;
    return rezultat;
}
```

Funkcija je definirana:

- Tipom povratne vrijednosti
- Imenom (predlaže se CamelCase)
- Parametrima koje prima (tip parametra i koliko ih ima)

Iznimke

Kao i u drugim programskim jezicima, C# ima razvijen mehanizam rukovanja iznimkama. Iznimka se može dogoditi uslijed nepredviđenih ulaznih parametara, pogreške u kodu i sl. Iznimke se mogu baciti i uloviti, ali svaka iznimka koju možemo baciti ili uloviti nasljeđuje iz bazne klase `Exception`. U samom .NET radnom okviru ima niz već postojećih iznimki koje se mogu iskoristiti u nizu situacija, ali moguće je i kreirati vlastitu iznimku – to je ni manje ni više nego nova klasa koja nasljeđuje iz klase Exception ili iz bilo koje njoj izvedene klase. Za bacanje iznimki koristimo ključnu riječ **throw**, dok za hvatanje iznimki koristimo kombinaciju **try-catch-(finally)**.

Sučelja (interface)

Sučelje je skup pravila kojim definiramo kako izgleda ili koja ponašanja sadrži neka klasa koja ga implementira. Primjerice, klasa koja implementira sučelje `IDisposable` sigurno sadrži metodu `Dispose`. Ovo sučelje također indicira da klasa koristi resurse koje bi trebalo osloboditi nakon što smo završili s korištenjem te klase. Prešutni dogovor u C# je da sučelja počinju slovom „I" (od Interface), te ih je na taj način lako uočiti i razlikovati od „običnih" klasa.

Kolekcije (generics)

Gotovo svaka poslovna aplikacija će morati voditi evidenciju (pamtiti) informacije o nizu različitih podataka. U tu svrhu koriste se razne kolekcije koje

nam omogućavaju niz funkcionalnosti za rukovanje ovisno o konkretnoj potrebi i kontekstu. U C# najčešće se koriste strogo tipizirane kolekcije (generics) – to su kolekcije koje mogu pamtiti samo određene tipove podataka, tj. objekte neke specifične klase.

- **List** – kolekcija u koju se može spremiti bilo kakav objekt. Ovo je primjer netipizirane kolekcije. Elementima liste se može pristupati preko uglatih zagrada i indexa: `mojaLista[5]` → vraća 6. element liste. Povratna vrijednost je tipa `object`.
- **List<int>** – lista cjelobrojnih vrijednosti, ne možemo dodati ništa osim cjelobrojne (int) vrijednosti. Primjer: `int x = mojaIntLista[2];`
- **List<ContactData>** – lista objekata tipa ContactData
- **Dictionary<int, string>** – kolekcija u kojoj su vrijednosti spremljene pod nekim ključem. U ovoj kolekciji ključ je cjelobrojna vrijednost, a vrijednost je string. Pristupamo na način: `mojDict[2] = "Dva"`

Najčešće korištena klasa je upravo `List<XX>`, gdje je XX odgovarajući tip.

Petlje i enumeriranje kolekcije

Da bi pregledali elemente neke kolekcije, moramo koristiti neku vrstu petlje. U C# najčešće se koristi **foreach** petlja koja prolazi po svim elementima neke kolekcije. Primjer takve petlje nalazi se u implementaciji metode `PrebrojiKolikoJeUListi` iznad.

C# - napredni koncepti

U narednom poglavlju bit će obrađeni napredni koncepti u C# jeziku – konkretnije lambda izrazi i LINQ mehanizam. Stabla izraza (expression trees) su posebne strukture u C# jeziku koje omogućavaju definiranje koda koji će tek kasnije biti izvršen. LINQ koristi stabla izraza za manipulaciju kolekcijama, gdje se pomoću stabla izraza definira upit koji će tek naknadno biti izveden. U ovom poglavlju bit će obrađene osnove LINQ izraza za manipulacije kolekcijama te `Func<>`, `Predicate<>` i `Action<>` objekti kojima se definira funkcija u obliku varijable.

Lambda izrazi i LINQ

Lambda izrazi se u osnovi mogu poistovjetiti s anonimnim ili inline funkcijama. Ideja manipulacije kolekcijama kroz LINQ je da svaka kolekcija, pozivom neke LINQ naredbe se transformira u novu kolekciju.

Lambda izrazi

Pomoću lambda izraza se definira anonimna funkcija ili stablo izraza koje se koristi pri izvršavanju primjerice LINQ upita. Sintaksa lambda izraza je sljedeća:

```
p => p.Id < 3
```

U gornjem izrazu, prvi parametar `p` (s lijeve strane operatora `=>`) označava jedan ulazni objekt kolekcije. S desne strane operatora `=>` se nalazi izraz koji koristi objekt `p`. Povratna vrijednost izraza s desne strane je `bool`. Gornji izraz je ekvivalentan ovom:

```
p =>
{
    if (p.Id < 3)
        return true;
    return false;
}
```

LINQ – naredba where

Vjerojatno najčešće korištena metoda prilikom manipulacija kolekcijama – služi za filtriranje i obradu samo određenih podataka.

```
var quizesWithLowId = listaKvizova
    .Where(p => p.Id < 3);
```

Lambda izraz – kao rezultat se vraćaju samo oni elementi iz kolekcije za koje je zadovoljen izraz, tj. za koje funkcija vraća `true`. Naredba `Where` se poziva nad kolekcijom objekata (primjerice `List<Quiz>`).

LINQ – naredba ToList

Svaku kolekciju možemo eksplicitno pretvoriti u listu pozivom `ToList()` metode koja će stvoriti novi `List<>` objekt i sve elemente kolekcije prebaciti u listu.

LINQ – naredbe First, Single, FirstOrDefault i SingleOrDefault

Gore navedene naredbe služe za izdvajanje samo jednog elementa iz kolekcije. Razlike:

- **First** – ukoliko nema niti jednog elementa u kolekciji, poziv naredbe baca iznimku
- **FirstOrDefault** – ukoliko nema niti jednog elementa, vraća default vrijednost (za klase `null`, za `int` vrijednost `0`)
- **Single** – ukoliko ima više od jednog elementa ili je kolekcija prazna, baca iznimku
- **SingleOrDefault** – slično kao Single, no vraća default vrijednost umjesto iznimke

LINQ – naredba OrderBy i OrderByDescending

Koristi se ukoliko je potrebno poredati kolekciju. `OrderByDescending` primjenjuje poredak u obrnutom redoslijedu od `OrderBy` poziva. Sve LINQ naredbe se mogu zajedno kombinirati.

LINQ – naredba Count

Kao rezultat vraća broj elemenata u kolekciji. Može se kombinirati s ostalim LINQ upitima i naredbama.

LINQ – podupiti

Čest je slučaj da je potrebno izdvojiti elemente kolekcije na temelju kolekcija koje sadrže. Unutar bilo kojeg LINQ upita, može se postaviti dodatni upit nad bilo kojom kolekcijom definiranom nekim svojstvom.

Func, Predicate i Action objekti

Slično kao što su pokazivači na funkcije u C/C++, u C# je uveden novi set objekata koji omogućavaju spremanje funkcije u varijablu, te kasnije pozivanje te funkcije s željenim parametrima i dohvaćanje rezultata. Primjerice, funkcija `GetRandomOperation()` vraća objekt `Func<int,int,int>`. Taj objekt predstavlja funkciju koja ima dva parametra tipa `int` i povratnu vrijednost tipa `int`. Zadnji parametar predloška je uvijek tip povratne vrijednosti funkcije.

Primjer – Program.cs:

```
namespace LINQHelperApp.Console
{
    class Program
    {
        static void Main(string[] args)
        {
            var calc = new Calculator();
            Func<int,int,int> fOp = calc.GetRandomOperation
();
            int x = 10, y = 5;
            int rez = fOp(x, y);
            System.Console.WriteLine(rez);
            System.Console.ReadKey();
        }
    }
}
```

Varijabla `fOp` je objekt koji tretiramo kao funkciju – pozivamo ga kao `fOp(x, y)`.

Primjer – Calculator.cs (s imenovanim metodama):

```
public class Calculator
{
    public Func<int, int, int> GetRandomOperation()
    {
        var rand = new Random();
        switch (rand.Next() % 3)
        {
            case 0: return Summation;
            case 1: return Substraction;
            case 2: return Multiplication;
        }
        return null;
    }

    public int Summation(int a, int b) => a + b;
    public int Substraction(int a, int b) => a - b;
```

```
    public int Multiplication(int a, int b) => a * b;
}
```

Primjer – Calculator.cs (s inline lambda izrazima):

```
public class Calculator
{
    public Func<int, int, int> GetRandomOperation()
    {
        var rand = new Random();
        switch (rand.Next() % 3)
        {
            case 0:
                return (a, b) => { return a + b; };
            case 1:
                return ((a, b) => a - b);
            case 2:
                return (a, b) => a * b;
        }
        return null;
    }
}
```

Moguće je i ovakav objekt proslijediti kao parametar nekoj drugoj metodi:

```
class Program
{
    static void PrintOperation(int x, int y, Func<int, int,
int> op)
    {
        int rez = op(x, y);
        System.Console.WriteLine(rez);
    }

    static void Main(string[] args)
    {
        var calc = new Calculator();
        Func<int,int,int> fOp = calc.GetRandomOperation();
```

```
        PrintOperation(10, 5, fOp);
        System.Console.ReadKey();
    }
}
```

LINQ funkcije koriste upravo `Func<>`, `Action<>` i `Predicate<>` objekte za obradu kolekcija:

- **Predicate<>** – isto što i `Func<>`, ali s povratnom vrijednošću `bool`
- **Action<>** – isto što i `Func<>`, ali bez povratne vrijednosti

---

Async arhitektura

Ključne naredbe **async** i **await** omogućavaju izuzetno lagano korištenje pozadinskih dretvi za izvršavanje procesa koji bi inače potrošili prijeko potrebne resurse. Postoje i performansni benefiti od korištenja async arhitekture u odnosu na sinkronu – najčešće u desktop/mobile okruženju gdje je korištenje UI dretve za dulje operacije nepovoljno.

Osim u desktop/klijentskim aplikacijama, async postaje bitan u kontekstu velikog broja paralelnih zahtjeva na server, jer ako pozive na bazu podataka odrađuje pozadinska dretva, oslobađaju se resursi za odrađivanje „običnih" zahtjeva za to vrijeme.

Klasa Task

U C# async arhitekturi, ključnu ulogu ima klasa `Task`, kao i ključna riječ `async`.

```
Task t1 = Task.Run(() =>
{
    Console.WriteLine($"Sleeping started");
    Thread.Sleep(1000);
    Console.WriteLine($"Sleeping completed");
});
Console.WriteLine($"Waiting on task..");
t1.Wait();
```

Opis izvođenja:

1. Kreiramo Task na način da mu definiramo što radi – to je primjer `Action` objekta iz prošlog poglavlja
    - Pozivom `Task.Run()` se taj task automatski pokreće
    - Spremamo ga u varijablu `t1`. U ovom trenutku task se već počeo izvršavati (vjerojatno)
2. Main program nastavlja daljnje izvođenje
3. Nakon toga zaustavljamo daljnje izvođenje dok god Task t1 ne završi: pozivom `t1.Wait()`
    - Alternativa `t1.Wait` bi bila statička metoda `Task.WaitAll(t1)` koja omogućava čekanje više taskova odjednom

Async metode

Kada bi htjeli postići istu funkcionalnost pomoću async metode:

```
private static async Task DoSomeSleepingAsync()
{
    Console.WriteLine($"Sleeping started");
    await Task.Delay(1000);
    Console.WriteLine($"Sleeping completed");
}

static void Main(string[] args)
{
    Task t1 = DoSomeSleepingAsync();
    Console.WriteLine($"Waiting on task..");
    t1.Wait();
    ...
}
```

Razlika je u korištenju `await Task.Delay()` umjesto `Thread.Sleep()`. Analiza izvođenja:

1. U Main metodi se kreira Task t1 pozivom async metode
    - Metoda `DoSomeSleepingAsync` ima definiranu povratnu vrijednost tipa `Task`, ali ne vraća nikakav rezultat (nema `return` naredbe). To odrađuje ključna riječ `async`.

2. Izvršava se prva linija unutar async metode
3. Nakon toga se dolazi do **await** naredbe. U tom trenutku se interno kreira nova dretva (thread) i izvršava spavanje od 1000ms. Za to vrijeme, u Main funkciji se nastavlja izvođenje paralelno
4. Kod poziva `t1.Wait()` se čeka da gornji Task završi

Generalno, često će async metode imati sufiks „Async" tako da ih je vrlo lako prepoznati. Razne popularne biblioteke će često imati istu metodu napisanu u sinkronom kontekstu i async kontekstu.